Calibrated Perplexity for Robust AI-Generated Text Detection Across Languages and Genres

Vladimir Chaikin
MIPT
Moscow, Russia
email@phystech.edu
& *AnastasiaVoznyuk*
MIPT
Moscow, Russia
email@phystech.edu

March 22, 2026

Abstract

This paper invistigates the problem of robust AI-generated text detection under distribution shift. While simple perplexity is a simple solution for such detection, its oppurtunities lose significantly on out-of-domain data. We propose a novel method - calibrated perplexity - which normalizes raw perplexity scores using domain-specific language model baselines. This approach constructs a stable feature space less sensitive to text length, genre, and language variations. Experiments across multiple domains and languages are going to demonstrate improved in-domain and out-of-domain detection accuracy compared to standard perplexity-based methods.

Keywords: Calibrated perplexity Cross-domain detection Robustness Machine-generated text Text classification Multilingual detection Genre adaptation Statistical methods Language modeling

1 Introduction

Fast improvement and widespread accessibility of Large Language Models (LLMs) have destroyed the lines between human-written and machine-generated text. While this technology offers benefits, it also presents significant disadvantages, including academic dishonesty, disinformation, and the fall of trust in digital content [Xu and Sheng(2024), Hans et al.(2024)]. This has created an inevitable need for robust methods to detect AI-generated content.

Current detectors often rely on fine-tuned classifiers or statistical signatures taken from the text [Gehrmann et al.(2019)Gehrmann, Strobelt, and Rush]. Use-

ful and computationally efficient feature is perplexity, a measure of how "surprising" a text is to a language model. The assumption is that LLMs, which are trained to maximize likelihood, will generate text with lower perplexity compared to human writing. As it is shown in [Bakhteev et al.(2022)], a significant change comes from distribution shift. The perplexity of a text is not an absolute value; it is highly dependent on the domain (e.g., news, scientific articles, social media) and language it was trained on. A model fine-tuned for one genre often fails on another, causing unreliable detection in real world, where the target domain is unknown.

Recent works have attempted to solve this by developing zero-shot detectors that are more robust to unseen prompts or models. For instance, Binoculars [Hans et al.(2024)] uses a ratio of scores from two different LLMs for calibration. Similarly, research on multilingual and cross-domain detection highlights the necessity of adapting methods to diverse data distributions [Gritsay et al.(2023)Gritsay, Grabovoy, Kildyakov, and Chekhovich]. Despite these advances, a method for calibrating perplexity for each domain itself to make it robust across different genres and languages remains still not-explored.

In this paper, we face the challenge of domain shift and try to avoid its effect on detection. We propose a novel method called calibrated perplexity, which normalizes perplexity scores against domain-specific baselines. The final goal is not to find out one universal calibrated perplexity score, but to get a set of parameters to have an opportunity to learn fastly new zero-shot models. Our approach includeses training KenLM n-gram models on text from multiple domains and using these models to compute a relative, rather than absolute, measure of text predictability. The core novelty of our work lies in transforming a domain-dependent score into a stable feature for classification. To check our method, we conduct experiments across multiple domains and languages, comparing in-domain and out-of-domain detection.

2 Problem statement

Let $\mathbf{W}$ be our alphabet, tokens consists from characters from that alphabet.

Let

$$\mathbb{D} = \left\{ \left[t_j \right]_{j=1}^{n} | t_j \in \mathbf{W}, n \in \mathbb{N} \right\}$$

be the space of our documents.

We have a set of N documents

$$\mathbf{D} = \bigcup_{i=1}^{N} D^i, D^i \in \mathbb{D}$$

We have K classes of our text-generative models, that generated fragments of text in $\mathbf{D}$.

We have $K + 1$ labels, that we use for multiclass classification, where 0 is the label of human-written fragment, $\{1...K\}$ are the labels that represent corresponding language model, let $\mathbf{C}$ be our set of labels.

Our model ϕ is a superposition of two mappings, $\mathbf{f}$ and $\mathbf{g}$. Mapping $\mathbf{f}$ is responsible for text segmentation. Mapping $\mathbf{g}$ is responsible for classifying obtained fragments.

$$\phi : \mathbf{g} \circ \mathbf{f},$$

$$\phi : \mathbb{D} \to \mathbb{T}, \qquad \mathbb{T} = \left\{ \left[t_{s_j}, t_{f_j}, C_j \right]_{j=1}^{J} | t_{s_j} = t_{f_{j-1}}, s_j \in \mathbb{N}_0, f_j \in \mathbb{N}, C_j \in \mathbf{C} \right\},$$

where J is a number of fragments in segmentation of the document, t_{s_j} is starting index of the j-th fragment, t_{f_j} is finishing index of the j-th fragment, C_j is class of the j-th fragment.

To estimate the statistical properties of text fragments, we use a character-aware n-gram language model, KenLM [Chen and Goodman(1996)]. Let a fragment of text be represented as a sequence of tokens $[t_a, t_{a+1}, ..., t_b]$, where $t_i \in \mathbf{W}$. KenLM provides a function $\mathcal{K}$ that computes the log-probability of this sequence under an n-gram model trained on a specific corpus.

Formally, we define a language model $\mathcal{M}_\mathcal{T}$ trained on a text corpus $\mathcal{T} \subset \mathbb{D}$. For a given text fragment $F = [t_a, ..., t_b]$, the model estimates its probability as the product of conditional probabilities of each token given its $n-1$ predecessors:

$$P_{\mathcal{M}_\mathcal{T}}(F) = \prod_{i=a}^{b} P_{\mathcal{M}_\mathcal{T}}(t_i \mid t_{i-1}, ..., t_{i-n+1}). \tag{1}$$

KenLM efficiently implements this estimation using modified Kneser-Ney smoothing, which is particularly effective for handling sparse n-gram occurrences.

The **perplexity** of a fragment F is then defined as the inverse geometric mean of the token probabilities:

$$\mathrm{PPL}_{\mathcal{M}_\mathcal{T}}(F) = P_{\mathcal{M}_\mathcal{T}}(F)^{-\frac{1}{|F|}} = 2^{-\frac{1}{|F|}\log_2 P_{\mathcal{M}_\mathcal{T}}(F)}, \tag{2}$$

where $|F| = b - a + 1$ is the length of the fragment.

This metric quantifies how "surprising" the fragment is to the model $\mathcal{M}_\mathcal{T}$; lower perplexity indicates that the fragment is more predictable based on the training corpus $\mathcal{T}$. In our work, we train multiple KenLM models $\{\mathcal{M}_{\mathcal{T}_d}\}$ on different domains $\mathcal{T}_d$ (e.g., news, science, social media). For a given fragment F, we compute a set of raw perplexities $\{\mathrm{PPL}_{\mathcal{M}_{\mathcal{T}_d}}(F)\}$. The subsequent mapping $\mathbf{g}$ then uses these values, along with the segmentation information from $\mathbf{f}$, to perform the final classification.

Contributions. Our contributions can be summarized as follows:

- We present...
- We demonstrate the validity of our theoretical results through empirical studies...
- We highlight the implications of our findings for...

Outline. The rest of the paper is organized as follows...

3 Related Work

Topic #1. TODO
Topic #2. TODO

4 Preliminaries

4.1 General notation

In this section, we introduce the general notation used in the rest of the paper and the basic assumptions.

4.2 Assumptions

TODO

5 Method

6 Experiments

6.0.1 Experiment Planning

Goal of the Computational Experiment The goal of the computational experiment is to evaluate the effectiveness of *perplexity-based detection* of AI-generated text using a KenLM language model trained on a balanced dataset of human and machine-generated texts. We try to assess the detection performance on an in-domain test set and to compare different threshold selection strategies (accuracy, F1-score, and ROC-AUC) for the binary classifier.

Data Description We use the **AINL dataset** (`train-kenlm.csv` and `dev_full.csv`), an open-access collection of Russian-language texts with annotations indicating whether a text is human-written or generated by a language model.

Data statistics:

Split	Source	Human Texts	Machine Texts (llama-3.3-70b)	Total
Train	train-kenlm.csv	8,769	8,700	35,158
Validation	dev_full.csv	∼1,100	∼1,100	∼2,200
Test	dev_full.csv	∼1,100	∼1,100	∼2,200

Table 1: Data statistics by split.

Data description:

- Texts are in Russian language
- Human texts are labeled as 'abstract'

- Machine texts are labeled with the model name ('llama-3.3-70b')
- All texts are preprocessed: filtering empty and short texts (< 100 characters)

Algorithm Configuration The base algorithm is **KenLM**, an n-gram language model with Kneser-Ney smoothing. We train a trigram model (order = 3) on the training data. For classification, we use a simple threshold-based detector: a text is classified as machine-generated if its perplexity is below a threshold.

Key parameters:

- Model order: 3
- Smoothing: Kneser-Ney
- Threshold selection: optimal value on validation set

Experimental Plan

1. **Data splitting:**
   - Train: all human + all llama-3.3-70b texts from `train-kenlm.csv`
   - Validation: 50% of human + 50% of llama-3.3-70b from `dev_full.csv`
   - Test: remaining 50% of human + 50% of llama-3.3-70b from `dev_full.csv`
2. **Model training:**
   - Train KenLM on `train_texts.txt` (human + llama texts)
   - Compute perplexity for all texts
3. **Threshold selection (on validation set):**
   - Find optimal threshold maximizing Accuracy
   - Find optimal threshold maximizing F1-score
   - Find optimal threshold by ROC-AUC
4. **Evaluation (on test set):**
   - Apply each threshold to test data
   - Compute Accuracy, F1-score, ROC-AUC
   - Compare different threshold strategies
5. **Analysis:**
   - Compare validation vs test performance
   - Analyze distribution of perplexity for human and machine texts

Expected Tables and Figures **Short list:**

1. Table: Optimal thresholds per metric (Accuracy, F1, ROC-AUC)
2. Figure: Distribution of perplexity (human vs machine) on test set

Long list:

Figure/Table	Description
Fig. 1	Accuracy vs threshold curve (X: threshold, Y: accuracy)
Fig. 2	F1-score vs threshold curve (X: threshold, Y: F1)
Fig. 3	Distribution of perplexity with F1-optimal threshold (histogram, human/machine)
Fig. 4	Test set perplexity distributions (histogram, density)
Fig. 5	Test set boxplot (human vs machine)
Fig. 6	ROC curve on test set (X: FPR, Y: TPR)
Fig. 7	Confusion matrix (heatmap)
Fig. 8	Bar chart: validation vs test metrics comparison (F1, Accuracy, ROC-AUC)
Fig. 9	Bar chart: prediction distribution by class
Table 1	Data statistics (splits, counts)
Table 2	Optimal thresholds on validation (F1, Accuracy, Youden)
Table 3	Test results with F1-optimized threshold (Accuracy, F1, ROC-AUC)
Table 4	Comparison: validation vs test metrics (F1, Accuracy, ROC-AUC)

Table 2: Expected tables and figures.

6.0.2 R: Preliminary Report

Perplexity Statistics The perplexity values for validation data show clear separation between classes:

Class	Mean Perplexity
Human	10.97
Machine (llama-3.3-70b)	5.98

Table 3: Perplexity statistics on validation set.

Threshold Selection The optimal threshold maximizing F1-score on validation is:

$$\text{threshold}_{\text{F1}} = 7.94$$

At this threshold, validation metrics are:

- Accuracy: 0.8776
- F1-score: 0.8811
- ROC-AUC: 0.9298

Test Results Applying the F1-optimized threshold to the test set yields:

Metric	Value
Accuracy	0.8640
F1-score	0.8685
ROC-AUC	0.9306

Table 4: Test set results with F1-optimized threshold.

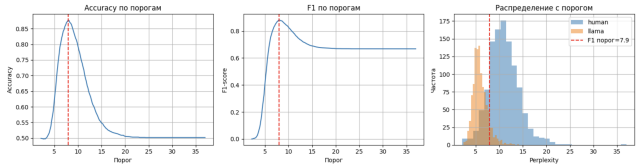
Metric	Validation	Test	Difference
F1-score	0.8811	0.8685	-0.0126
Accuracy	0.8776	0.8640	-0.0136
ROC-AUC	0.9298	0.9306	+0.0008

Table 5: Comparison of validation and test metrics.

Validation vs Test Comparison The small difference between validation and test metrics confirms the stability of the model and the representativeness of the data split.

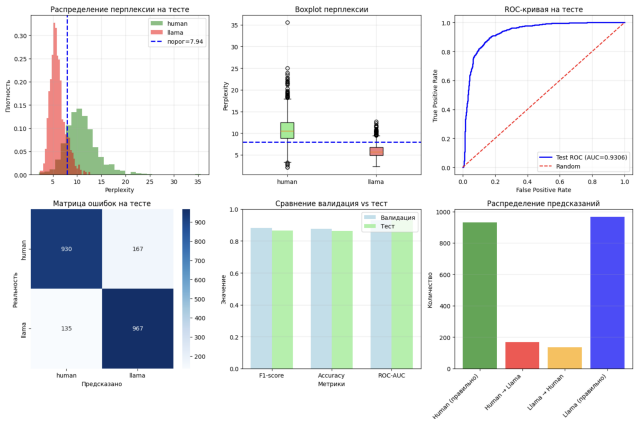
	Predicted Human	Predicted Machine
Actual Human	930	167
Actual Machine	135	967

Table 6: Confusion matrix for test set (F1-optimized threshold).



Interpretation The experimental results demonstrate that:

1. Perplexity effectively separates human-written and machine-generated texts (validation AUC = 0.93).
2. The F1-optimized threshold (79.4) provides balanced classification performance.
3. Test set metrics closely match validation metrics (F1 difference = -0.0126), confirming model stability.



6.0.3 B: Basic Code

The basic algorithm implementation is available in the notebook `exp1.ipynb` in the repository. The key steps include:

1. KenLM installation and compilation
2. Data loading from `train-kenlm.csv` and `dev_full.csv`
3. Perplexity calculation for each text
4. Threshold optimization on validation set (by F1-score)
5. Evaluation on test set

Key code sections:

- `get_perplexity()`: computes perplexity for a single text
- Threshold search loops over possible values
- Metrics calculation using `sklearn`

The code is modular and reproducible. The basic model (trigram KenLM with Kneser-Ney smoothing) runs efficiently on CPU, processing approximately 1000 texts per minute.

Repository structure:

```
experiments/
    exp1.ipynb          # main notebook with experiments
    train-kenlm.csv     # training data
    dev_full.csv        # validation and test data
```

7 Discussion

TODO

8 Conclusion

TODO

References

[Xu and Sheng(2024)] Z. Xu and V. S. Sheng. Detecting ai-generated code assignments using perplexity of large language models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(21):23155–23162, 2024.

[Hans et al.(2024)] Abhimanyu Hans et al. Spotting llms with binoculars: Zero-shot detection of machine-generated text. *arXiv preprint arXiv:2401.12070*, 2024.

[Gehrmann et al.(2019)Gehrmann, Strobelt, and Rush] Sebastian Gehrmann, Hendrik Strobelt, and Alexander M. Rush. Gltr: Statistical detection and visualization of generated text. *arXiv preprint arXiv:1906.04043*, 2019.

[Bakhteev et al.(2022)] O. Bakhteev et al. Automatic detection of machine generated texts: Need more tokens. *2022 Ivannikov Memorial Workshop (IVMEM)*, 2022.

[Gritsay et al.(2023)Gritsay, Grabovoy, Kildyakov, and Chekhovich] G. M. Gritsay, A. V. Grabovoy, A. S. Kildyakov, and Yu. V. Chekhovich. Artificially generated text fragments search in academic documents. *Doklady Mathematics*, 108:S434–S442, 2023.

[Chen and Goodman(1996)] Stanley F. Chen and Joshua Goodman. An empirical study of smoothing techniques for language modeling. *arXiv preprint cmp-lg/9606011*, 1996.

A Appendix / supplemental material

A.1 Additional experiments / Proofs of Theorems

TODO